

التقرير الفني النهائي لمشروع الحوسبة السحابية والحاويات

REST API و Docker نظام إدارة المهام باستخدام

اسم المقرر :حوسبة سحابية

Task Management API :اسم المشروع

:أعضاء الفريق

1. [اسم عضو الفريق الأول]

2. [اسم عضو الفريق الثاني]

3. [اسم عضو الفريق الثالث]

4. [اسم عضو الفريق الرابع]

الدكتور :اسم الدكتور

الجامعة :اسم الجامعة

2026

Table of Contents

1	1. الملخص التنفيذي (Executive Summary)	2
2	2. أهداف المشروع	2
3	3. نظرة عامة على النظام	2
4	4. معمارية النظام (System Architecture)	3
5	5. REST API وCRUD	3
6	6. PostgreSQL قاعدة البيانات	3
7	7. Virtualization vs Containerization	3
8	8. Dockerfile وتحسين الطبقات	4
9	9. Docker Compose	4
10	10. Network Architecture	4
11	11. Nginx Reverse Proxy	4
12	12. Storage Architecture وPersistence	4
13	13. Persistence إثبات	5
14	14. Security Practices	5
15	15. Failure Mode Scenarios	5
16	16. Testing Evidence	5
17	17. Docker Hub / Registry & Versioning	5
18	18. Reproducible Deployment	6

19	19. Git وGitHub.....	6
20	20. Team Contributions	6
21	21. خطوات تشغيل المشروع	6
22	22. الخاتمة	6
23	23. جدول Verification Status	6

1 (Executive Summary) الملخص التنفيذي 1

يهدف المشروع إلى حل (Task Management API) يقدم هذا التقرير تفصيلاً فنياً لتصميم وبناء نظام إدارة المهام (Deployment) وتعقيد النشر (Environment Inconsistency) مشكلة عدم توافق بيئات التشغيل يتكون النظام من Docker. ممثلة في (Containerization) من خلال استخدام تقنية الحاويات (Complexity) والتي تتواصل مع قاعدة Express وإطار العمل Node.js مبرمجة باستخدام (REST API) واجهة برمجة تطبيقات ولضمان الأمان وتوجيه الطلبات بشكل PostgreSQL. ممثلة في (Relational Database) بيانات علائقية لربط هذه Docker Compose استخدم Nginx بواسطة (Reverse Proxy) مركزي، تم استخدام وكيل عكسي وإثبات استمرارية (Failure Modes) المكونات معاً لتشغيلها كبيئة واحدة معزولة، وتمت محاكاة سيناريوهات الفشل (Docker Hub) إلى السجل العام (Docker Image) وأخيراً، تم رفع صورة النظام. (Persistence) البيانات لضمان سهولة التوزيع.

2 أهداف المشروع 2

تطبيق عمليات REST API يهدف هذا المشروع إلى تحقيق عدة مخرجات تعليمية وهندسية أساسية، تشمل :- بناء عزل كل تقنية في (Containerization) التشغيل المعزول - (CRUD) الإنشاء والقراءة والتحديث والحذف لتوحيد التشغيل - توجيه الشبكة Docker Compose حاوية مستقلة - إدارة الحاويات المتعددة: استخدام (Network Isolation) لإخفاء المعمارية الداخلية - العزل الشبكي Nginx استخدام (Reverse Proxy): تخزين البيانات بأمان (Persistent Storage) حماية الخدمات الخلفية من الوصول المباشر - الاستمرارية التحقق من قدرة النظام على التعامل مع (Failure Mode Testing) لحمايتها من زوال الحاويات - اختبار الفشل التشغيل - (Docker Registry) رفع الصور إلى السجل (Versioning) التوقف المفاجئ - إدارة الإصدارات القدرة على تشغيل المشروع بخطوة واحدة على أي جهاز (Reproducible Deployment) المتطابق.

3 نظرة عامة على النظام 3

Client → Nginx → Node.js / Express API → PostgreSQL ويتدرج كالتالي (Client) من العميل (Request Flow) يبدأ مسار الطلب

1. Nginx: يستقبل الطلب من خارج النظام ويوجهه للداخل.
2. Node.js API: يستقبل الطلب الموجه، يحلل البيانات، وينفذ منطق العمل (Business Logic).
3. PostgreSQL: ينفذ استعلام قاعدة البيانات لحفظ أو جلب المهام ويُرجع النتيجة ليتم تصديرها للعميل.

4. معمارية النظام (System Architecture)

يعمل على المنفذ: **Nginx** - معزولة كلياً (3-Tier Architecture) يعتمد المشروع على معمارية ثلاثية الطبقات يعمل على المنفذ: **API** - ليكون نقطة الدخول الوحيدة 8080:80 (Host) داخل الحاوية. تم ربطه بمنفذ المضيف 80 ask- يتصل به عبر العنوان الداخلي Nginx. للمضيف (Publish) ولا يتم نشره، Docker داخل شبكة 3000 وهو معزول Docker، يعمل على المنفذ 5432 داخل شبكة: **PostgreSQL** - management-api:3000. ask-management-db:5432 عبر العنوان API تماماً عن المضيف ولا يتصل به إلا الـ

5. REST API وCRUD

العميل وترد عليها (HTTP Requests) هي حلقة الوصل التي تستقبل طلبات (REST API) واجهة برمجة التطبيقات (HTTP Responses). الأساسية كالتالي CRUD تم تطبيق عمليات.

العملية	HTTP Method	Endpoint
إنشاء	POST	/api/tasks
قراءة	GET	/api/tasks
تعديل	PUT	/api/tasks/:id
حذف	DELETE	/api/tasks/:id

6. PostgreSQL قاعدة البيانات

تحت اسم (Database) لقوتها كقاعدة بيانات علانقية. يتكون النظام من قاعدة بيانات PostgreSQL تم اختيار مثل (Columns) يتكون من أعمدة (Row) كل مهمة تمثل صفاً. asks يسمى (Table) تحتوي على جدول askdb API ويرتبط بها الـ SQL يتم التواصل معها عبر أوامر. status، و itle، و id (Primary Key كمفتاح أساسي) ask-management-db:5432 عبر اسم الخدمة الداخلي.

7. Virtualization vs Containerization

لإثبات كفاءة معماريتنا، نوضح الفرق بين الأجهزة الافتراضية والحاويات:

الخاصية	Virtual Machine	Container
Guest OS	يتطلب نظام تشغيل كامل لكل آلة	لا يوجد، تشارك نواة المضيف (Host Kernel)
الموارد	مقطعة RAM/CPU ثقيلة وتستهلك	خفيفة جداً تستهلك موارد حسب الحاجة
Startup Time	دقائق	(Milliseconds) أجزاء من الثانية
Isolation	عزل قوي (Hardware Level)	عزل على مستوى العمليات (Process Level)
Portability	صعبة النقل (ملفات بحجم جيجابايت)	سهلة النقل والتوزيع (ميجابايت)

الخيار الأمثل لهذا المشروع لأنه يسمح بتشغيل الثلاث طبقات على نفس الجهاز باستهلاك موارد Docker يعتبر ضئيل وبسرعة فائقة.

8 وتحسين الطبقات 8. Dockerfile

API: المستخدم لبرمجة صورة الـ Dockerfile إليك ملف

dockerfile FROM node:24-alpine WORKDIR /usr/src/app COPY package*.json ./ RUN npm install --omit=dev COPY . . USER node EXPOSE 3000 CMD ["npm", "start"] - **FROM:** لتخفيف الحجم لpine يعتمد على صورة مصغرة: - **WORKDIR:** COPY package*.json ./ تم وضع **Layer Caching** - يحدد مجلد العمل داخل الحاوية: - **EXPOSE:** هذا يضمن عدم إعادة تنزيل المكتبات إذا تم تعديل الكود المصدري فقط، ... COPY قبل أمر pm install متبوعاً بـ (Least Privilege). لتعزيز الأمان ode إلى oot يحول المستخدم من: **USER** - مما يسرّع البناء يوثق أن المنفذ 3000 مستخدم داخلياً، ولكنه لا يفتحه على المضيف.

9 9. Docker Compose

ask-management-db - ask-management-api - ask-management-nginx healthcheck مع depends_on استخدمنا environment، إلا بعد جاهزية قاعدة البيانات تماماً. وُضعت المتغيرات الأمنية في قسم API عدم عمل الـ وقاعدة البيانات مخفيتين (API فقط) 8080:80 (لإبقاء الـ Nginx لحاوية ports تم تخصيص etworks. الجميع في تماماً.

10 10. Network Architecture

هذه الشبكة ask-management-network. تسمى Bridge Network يعتمد المشروع على شبكة من نوع بفضل (Service Discovery) واكتشاف الخدمات (Internal DNS) توفر خدمة نظام أسماء النطاقات الداخلي دون الحاجة ask-management-db التحدث مع قاعدة البيانات بمجرد استخدام اسمها API هذا، يمكن لحاوية الـ المتغير الخاص بها IP لمعرفة عنوان الـ

11 11. Nginx Reverse Proxy

يعمل Nginx كوكيل عكسي (Reverse Proxy). مسار الطلب الفعلي هو: Client → localhost:8080 → Nginx 80 → ask-management-api:3000 → PostgreSQL 5432 (Network Isolation) يحقق هذا أماناً عالياً (Traffic Routing). ويوفر نقطة دخول واحدة مركزية لتنظيم حركة المرور،

12 12. Storage Architecture و Persistence

هو نظام مؤقت يُمحي بمجرد حذف الحاوية . (Container filesystem) بشكل افتراضي، نظام ملفات الحاوية ask-management-db-data تدعى (Named Volume) لضمان بقاء البيانات، استخدمنا وحدة تخزين مسماة هذا يفصل دورة حياة البيانات عن دورة . /var/lib/postgresql/data وتم ربطها بمسار قاعدة البيانات حياة الحاويات

13 Persistence إثباتات 13

docker compose down جديدة ثم إيقاف النظام بأمر (Task) للتأكد من استمرارية البيانات، قمنا بإنشاء مهمة وطلب البيانات، وُجدت المهمة كما هي. تحذير: لا ينبغي docker compose up -d وعند تشغيل النظام مجدداً بـ. ويمسح البيانات نهائياً named volumes لأنه يقوم بتدمير الـ docker compose down -v أبداً استخدام

14 Security Practices 14

إخفاء المنافذ 3000 و - USER node. - بمستخدم غير جذري API ركزنا على الجوانب الأمنية من خلال :- تشغيل الـ .gitignore كمدخل خارجي وحيد . - استخدام ملفات Nginx عن المضيف تماماً . - الاعتماد على 5432 GitHub إلى (node_modules) أو الكبيرة (Secrets) لمنع رفع الملفات الحساسة dockerignore. و omit=dev. - والسجل . - تثبيت حزم الإنتاج فقط عبر

15 Failure Mode Scenarios 15

تمت محاكاة انهيار النظام للتأكد من قدرته على التعامل مع الأخطاء

التفسير	النتيجة	الحالة
يعمل لكنه لا يجد الخادم Nginx الخلفي.	502 Bad Gateway	API توقف
يعمل لكنه يفشل في تنفيذ API الـ الاستعلامات.	500 Internal Server Error	Database توقف
نقطة الدخول متوقفة كلياً.	Connection Refused	Nginx توقف
يعالج الطلب ويرد برمز خطأ API الـ سليم.	404 Not Found	غير موجود Task ID
يرفض البيانات لعدم مطابقتها API الـ للشروط.	400 Bad Request	بيانات خاطئة POST

16 Testing Evidence 16

GET يقوم تلقائياً بطلب pi/tests/test.js ملف - Automated Testing: - تم تنفيذ الاختبارات على مستويين DELETE، PUT و POST تم اختبار وظائف - Manual Runtime Testing: - API للتأكد من عمل الـ بالإضافة لسيناريوهات الفشل المختلفة، يدوياً أثناء التشغيل للتأكد من الموثوقية الشاملة

17 Docker Hub / Registry & Versioning 17

المستودع - Docker Hub. في موقع (Docker Registry) المشروع إلى السجل (Image) تم بناء ورفع صورة hishamdiv/task-management-api (Tags) تم استخدام علامات latest: - لتجديد إصدار مستقر لا يتغير 1.0.0: - (Versioning) ليكون مؤشراً لأحدث نسخة متوفرة

18. Reproducible Deployment

يمكن لأي شخص الآن إعادة تشغيل النظام بكفاءة وفي ثوانٍ عبر الأوامر `ext git clone [repository-url] cd task-management-api docker compose up -d` يضمن هذا التوافق الكامل عبر Docker Hub وصور Dockerfile الأجهزة المختلفة بفضل الـ

19. Git وGitHub

تم رفع `main` مثل (Branch) وحفظ الفروع (Commit) من خلال التزام Git تم تتبع التغييرات باستخدام نظام مما يعزز أمان الكود وسهولة الوصول إليه، GitHub على موقع (Remote) المشروع إلى الخادم البعيد.

20. Team Contributions

المسؤوليات	عضو الفريق
Backend Development & Containerization	[اسم عضو الفريق الأول]
Reverse Proxy & Security	[اسم عضو الفريق الثاني]
Database & Storage	[اسم عضو الفريق الثالث]
Testing & Documentation	[اسم عضو الفريق الرابع]

21. خطوات تشغيل المشروع

للتحقق العملي من النظام، يمكنك تنفيذ الأوامر التالية `powershell cd "C:\Users\NOMAN\Projects\task-management-api" docker compose config docker compose up -d --build docker compose ps Invoke-RestMethod -Uri "http://localhost:8080/api/tasks" -Method GET`

22. الخاتمة

تم بناء واجهة تطبيقات مستقرة Docker أثبت هذا المشروع نجاحه في تقديم معمارية قوية ومرنة باستخدام حاويات ومعزولة شبكياً ومحمية بواسطة وكيل عكسي. مع ضمان استمرارية البيانات وإدارة الانهيئات بطريقة سليمة، نكون قد حققنا كافة متطلبات النشر السحابي الحديث.

23. Verification Status جدول

العنصر	الحالة
Docker Compose	RUNTIME VERIFIED
API	RUNTIME VERIFIED
PostgreSQL	RUNTIME VERIFIED
Nginx	RUNTIME VERIFIED

العنصر	الحالة
Network Isolation	RUNTIME VERIFIED
Persistence	RUNTIME VERIFIED
CRUD	RUNTIME VERIFIED
Docker Hub	RUNTIME VERIFIED
Failure Modes	RUNTIME VERIFIED
Automated GET Test	RUNTIME VERIFIED
Full Automated CRUD Suite	NOT IMPLEMENTED / NOT CLAIMED